
title: Redaction Recovery via Named Entity Inference subtitle: System Specification for Neural Pipeline date: 2026-04-07

Redaction Recovery System Specification

1. Overview

Train a neural network to recover redacted named entities from context. Given text with `[REDACTED]` markers and surrounding context, predict the original entity name.

Constraint: Single device, 1-2GB VRAM. Run full pipeline end-to-end.

2. Corpus Specification

Source

- **Wikipedia:** English language dump
- **Rationale:** High signal density, rich cross-references, well-structured NE distribution

Size (Working Backwards from VRAM)

- **Target articles:** 500-1000
- **Raw text size:** 50-100 MB
- **Token count:** 10-20M tokens
- **Fits in VRAM:** Yes (batch size 8-16, sequence length 512)

Processing

- Extract plain text from Wikipedia markup
 - Remove boilerplate (infoboxes, references)
 - Filter articles < 500 tokens or > 5000 tokens
-

3. Named Entity Redaction

Entities in Scope

- Person names

- Place names (cities, countries, landmarks)
- Organization names
- Dates (if isolated as single entities)

Detection Method

- Rule-based: regex patterns for common NE formats
- Fallback: spaCy NER model for validation (optional for MVP)

Redaction Configuration

Parameter	Value	Rationale
Rate	2-5% of NEs per document	Sparse, recoverable from context
Span length	1-5 tokens	Single names or multi-token entities (e.g., "New York City")
Determinism	Seeded by doc_id hash	Reproducible train/test splits
Marker	[REDACTED]	Plain text, no special tokens

Example

Original: "Napoleon Bonaparte defeated the Austrians at Austerlitz."
Redacted: "[REDACTED] [REDACTED] defeated the Austrians at [REDACTED]."

4. Dataset Output Format

Format: JSONL (one record per line)

json

```
{
  "doc_id": "wikipedia_article_12345",
  "original": "Napoleon Bonaparte defeated the Austrians at Austerlitz.",
  "redacted": "[REDACTED] [REDACTED] defeated the Austrians at [REDACTED].",
  "redaction_spans": [
    {"start": 0, "end": 8, "entity": "Napoleon Bonaparte", "type": "PERSON"},
    {"start": 9, "end": 19, "entity": "Bonaparte", "type": "PERSON"},
    {"start": 59, "end": 70, "entity": "Austerlitz", "type": "PLACE"}
  ]
}
```

Fields

- `doc_id`: Unique document identifier (for deterministic splits)
- `original`: Full original text
- `redacted`: Text with `[REDACTED]` markers
- `redaction_spans`: List of redacted entities with metadata

Storage

- Output: `data/corpus.jsonl`
- Size estimate: 200–500 MB (compresses well with gzip)

5. Train/Test Split

Split	Ratio	Selection	Use
Train	80%	<code>hash(doc_id) % 10 < 8</code>	Model training
Test	20%	<code>hash(doc_id) % 10 >= 8</code>	Final evaluation

- Deterministic: same split on re-runs
 - No document leakage: entity appears in only one split
-

6. Model Input/Output Specification

Input Format

[Context] [REDACTED] [Context]

- **Left context:** Up to 256 tokens preceding the [REDACTED] marker
- **Redacted marker:** [REDACTED]
- **Right context:** Up to 256 tokens following the marker
- **Total sequence:** ≤ 512 tokens

Example Input

"Napoleon [REDACTED] defeated the Austrians at Austerlitz in 1805."

Output Target

"Bonaparte"

(Or: the original entity name as token sequence)

Model Type

- **Encoder:** DistilBERT (base, 66M parameters)
- **Head:** Linear projection to vocabulary logits (span prediction)
- **Inference:** Greedy decoding (top token) or beam search (k=3)

Why This Architecture

- Fits comfortably in 1-2GB VRAM
- Pre-trained on masked language modeling (similar task)
- Fast inference for paper deadline

7. Validation & Metrics

Primary Metric

- **Exact Match Accuracy:** Predicted entity == original entity (case-insensitive)

Secondary Metrics

- **Token F1:** Overlap between predicted and original token sequences
- **Entity Type Accuracy:** Did the model predict the right type of entity?

Test Set

- Held-out 20% of documents (by doc_id hash)
- No entity appears in both train and test

Baseline

- Simple context-based: "return most common entity in corpus near this context"
 - Target: Model should beat baseline by ≥ 10 percentage points
-

8. Implementation Constraints

Hardware

- **VRAM:** 1–2 GB (single device, no distributed training)
- **Disk:** 500 MB for corpus + 500 MB for checkpoints

Libraries

- **Data:** `datasets`, `jsonl`
- **NLP:** spaCy (for NER validation), tokenizers
- **ML:** PyTorch (or TensorFlow Lite)
- **Serialization:** PyTorch `.pt` format for models

Development Iteration

1. Data pipeline (Week 1)
 2. Model training (Week 1)
 3. Inference + evaluation (Week 2)
 4. Paper writeup (Week 2–3)
-

9. Deliverables

Code

- `scripts/download_corpus.py` - Fetch and process Wikipedia
- `scripts/redact_corpus.py` - Generate JSONL dataset
- `scripts/train.py` - Model training loop
- `scripts/evaluate.py` - Metrics on held-out set
- `scripts/infer.py` - CLI for inference on new documents

Data

- `data/corpus.jsonl` (or `.jsonl.gz`)
- `data/train.jsonl`, `data/test.jsonl` (splits)

Model

- `models/redaction_recovery.pt` - Trained DistilBERT + head

Paper

- Typst document: methods, results, discussion
 - Figures: redaction rate vs. accuracy, examples, error analysis
-

10. Success Criteria

- ☐ Pipeline runs end-to-end on single device (≤ 2 GB VRAM)
 - ☐ Exact match accuracy $\geq 40\%$ on held-out test set
 - ☐ Inference time ≤ 100 ms per redaction
 - ☐ Code is reproducible (deterministic seeds, logged config)
 - ☐ Paper is complete and preprint-ready
-

Appendix: Configuration Locked

These parameters do **not** change mid-development:

- Corpus source: Wikipedia (English)
- Redaction rate: 2-5% of NEs

- Span length: 1-5 tokens (named entities only)
- Output format: JSONL
- Train/test split: 80/20 by doc_id hash
- Model: DistilBERT + linear head
- VRAM budget: 1-2 GB